WOXSEN UNIVERSITY

School of Technology

Department of Computer Science and Engineering

FORENSITRACE

Endpoint Incident Reconstruction and Evidence Correlation

Digital Forensics Laboratory Mini-Project Report

Student Name	Rishvin Reddy
Roll Number	24WU0103006
Section	BIC - TIGERS
Programme	B.Tech Computer Science and Engineering
Specialization	Blockchain, IoT and Cybersecurity
Subject	Digital Forensics Laboratory
Faculty	Dr. Rajkumar Kalimuthu
Designation	Associate Professor & Head of the Department (HoD)
Institution	Woxsen University, School of Technology

Academic Year 2026–2027

October 2026

Project Scope and Evidence Notice

This report documents a functional software prototype for organizing and correlating digital-forensics evidence records. It includes a Python-based record validator, synthetic case records, automated tests, report exports, and four screenshots of the prototype interface.

Important scope limitation: The records and screenshots supplied with this report are synthetic demonstrations. They are not captured outputs from CrowdResponse, FTK Imager, or Autopsy, and this report does not claim that those external applications were executed against a real endpoint. No real forensic disk image or genuine external-tool result is represented as having been collected. The report distinguishes the working software prototype from the practical three-tool workflow that remains to be performed in an authorized lab environment.

Certificate

This is to certify that **Rishvin Reddy**, Roll Number **24WU0103006**, of the B.Tech Computer Science and Engineering programme, specializing in Blockchain, IoT and Cybersecurity, has prepared this mini-project report titled **ForensiTrace: Endpoint Incident Reconstruction and Evidence Correlation** as part of the Digital Forensics Laboratory coursework at Woxsen University, School of Technology.

Faculty In-charge	**Student**
Dr. Rajkumar Kalimuthu	Rishvin Reddy

Place: Woxsen University **Date:** October 2026

Declaration

I declare that this report describes the ForensiTrace software prototype and its included synthetic demonstration dataset. The implementation and automated tests are documented according to the work included in the project package. Synthetic examples are explicitly identified and are not presented as actual forensic findings.

I understand that digital-forensic tools must be used responsibly and only on systems, devices, accounts, and data for which appropriate authorization has been obtained. Any practical execution of CrowdResponse, FTK Imager, or Autopsy must be performed in a controlled, authorized environment, with evidence handling and limitations documented accurately.

Student Name: Rishvin Reddy

Roll Number: 24WU0103006

Signature: _____ **Date:** _____

Acknowledgement

I would like to acknowledge the guidance provided through the Digital Forensics Laboratory coursework at Woxsen University. The laboratory exercises introduced the use of forensic tools for system triage, evidence acquisition, and disk-image examination. This mini-project builds on those areas by organizing findings into a consistent case structure and documenting the limits of the available evidence.

I also acknowledge the importance of responsible evidence handling, reproducible procedures, and transparent reporting in digital-forensics work.

Abstract

ForensiTrace is a lightweight digital-forensics workflow prototype designed to organize and correlate records associated with live system triage, disk acquisition, and disk-image examination. The intended forensic workflow uses CrowdResponse, FTK Imager, and Autopsy. A Python utility validates the structure of evidence records, checks ISO-formatted timestamps, detects duplicate evidence identifiers, and exports JSON and CSV reports.

The supplied project includes a synthetic case containing five demonstration findings, a record-validation utility, automated tests, documentation, and four interface illustrations. The bundled tests confirm that the sample records pass validation and that duplicate evidence identifiers are rejected. These results validate the record-management software only; they do not represent the execution of the three external forensic tools. A practical investigation using actual tool outputs remains a separate step that must be conducted in an authorized test environment.

Contents

Project Scope and Evidence Notice i

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Aim 1

1.4 Objectives 1

1.5 Project Scope 2

2 Tools and Forensic Workflow 3

2.1 CrowdResponse: Live System Triage 3

2.2 FTK Imager: Acquisition and Integrity Documentation 3

2.3 Autopsy: Disk-Image Examination 3

2.4 Integrated Workflow 3

3 System Design and Implementation 5

3.1 Design Overview 5

3.2 Data Model 5

3.3 Validation Logic 6
3.4 Execution 6

4 Results and Testing **8**
4.1 Dataset Summary 8
4.2 Test Results 8
4.3 Evidence Correlation Example 9
4.4 Interpretation 10

5 Evidence Handling and Limitations **11**
5.1 Evidence Integrity 11
5.2 Chain of Custody 11
5.3 Limitations 11

6 Conclusion and Future Work **12**

A Project Execution and File Structure **13**
A.1 Project Structure 13
A.2 Reproduction Commands 13
A.3 Sample Finding 14

References **15**

List of Figures

2.1 ForensiTrace case overview illustration. The records shown are synthetic. . . . 4

3.1 Evidence inventory illustration showing synthetic records and source categories. 6

4.1 Correlation utility result illustration. PASS applies only to synthetic record validation. 9

4.2 Report-preview illustration demonstrating the separation of observation, interpretation, and limitation. 10

CHAPTER 1

Introduction

1.1 Background

Digital-forensics investigations frequently require information from more than one source. Live system triage can provide context about the state of a computer, forensic acquisition creates an image of a selected evidence source, and forensic examination tools help locate files and artifacts within that image. The outputs must be recorded and interpreted carefully to avoid confusing observations with assumptions.

ForensiTrace focuses on the organization and correlation layer of this workflow. It does not replace the specialist tools. Instead, it provides a common structure in which evidence identifiers, timestamps, source descriptions, observations, interpretations, and limitations can be documented.

1.2 Problem Statement

When multiple tools are used during an investigation, their outputs may be stored in different formats and locations. Without a consistent record structure, it becomes harder to track provenance, compare findings, and explain how a conclusion was reached. The project addresses this documentation problem with a small validation and reporting utility.

1.3 Aim

To design and implement a reproducible evidence-record organization and correlation prototype based on the three forensic workflow stages represented by CrowdResponse, FTK Imager, and Autopsy.

1.4 Objectives

- Define a case structure for documenting evidence and findings.

- Record the source, timestamp, artifact, observation, interpretation, and confidence for each finding.
- Validate required fields and ISO-formatted timestamps.
- Detect duplicate evidence identifiers.
- Export structured JSON and CSV reports.
- Document limitations and avoid unsupported claims about real forensic activity.

1.5 Project Scope

The implemented software validates and exports the supplied demonstration records. The package does not acquire disk or memory, invoke CrowdResponse, automate FTK Imager, or parse Autopsy's native case database. Those external applications form the intended practical workflow, but their execution is not claimed in this report.

CHAPTER 2

Tools and Forensic Workflow

2.1 CrowdResponse: Live System Triage

CrowdResponse is intended to collect configured system information from an authorized Windows test endpoint. For a practical run, the investigator should preserve the native output, note the configuration used, record the execution time, and document any errors or incomplete collection. A triage record is one evidence source and should not be treated as a complete explanation of an incident.

2.2 FTK Imager: Acquisition and Integrity Documentation

FTK Imager is intended to acquire a forensic image of a dedicated test disk and provide acquisition-related integrity information. A real lab record should identify the source, acquisition method, image format, output location, hash algorithm, hash value, verification outcome, and any errors. A successful image verification supports integrity within the documented scope; it does not prove that the source system was untampered with before acquisition.

2.3 Autopsy: Disk-Image Examination

Autopsy is intended to examine the acquired disk image. Depending on the evidence and enabled ingest modules, an examiner may inspect files, metadata, file types, extension mismatches, supported recent-activity artifacts, and keyword-search results. Only findings actually produced and reviewed in the practical session should be entered as real observations.

2.4 Integrated Workflow

1. Define the case scope and prepare an authorized Windows test environment.
2. Create and document benign test artifacts.

3. Run CrowdResponse and preserve its actual output.
4. Acquire the dedicated test disk with FTK Imager and document integrity verification.
5. Add the acquired image to Autopsy and examine relevant artifacts.
6. Record findings using unique evidence IDs.
7. Run the ForensiTrace validator and review its JSON and CSV exports.
8. Interpret the findings, document limitations, and prepare the report.

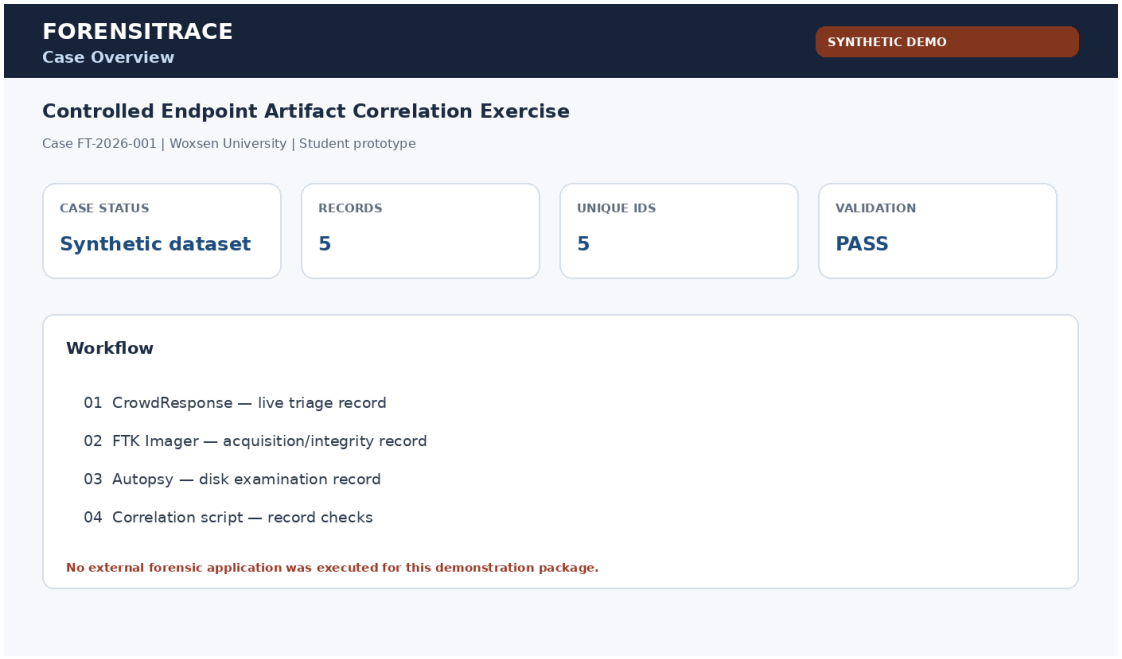


Figure 2.1: ForensiTrace case overview illustration. The records shown are synthetic.

CHAPTER 3

System Design and Implementation

3.1 Design Overview

The project uses JSON for case metadata and findings, Python for validation and correlation, and JSON/CSV for generated output. This approach avoids unnecessary dependencies and keeps the record structure inspectable. The application is designed to support a repeatable workflow without modifying the source evidence.

3.2 Data Model

Each finding contains the following fields:

- `evidence_id`: unique identifier for the record.
- `source_tool`: tool or source category associated with the record.
- `timestamp`: ISO-formatted timestamp.
- `artifact`: artifact or evidence item being documented.
- `observation`: direct description of what the record reports.
- `interpretation`: cautious explanation of its possible significance.
- `confidence`: confidence label or qualification.
- `hash_status`: integrity note when applicable.

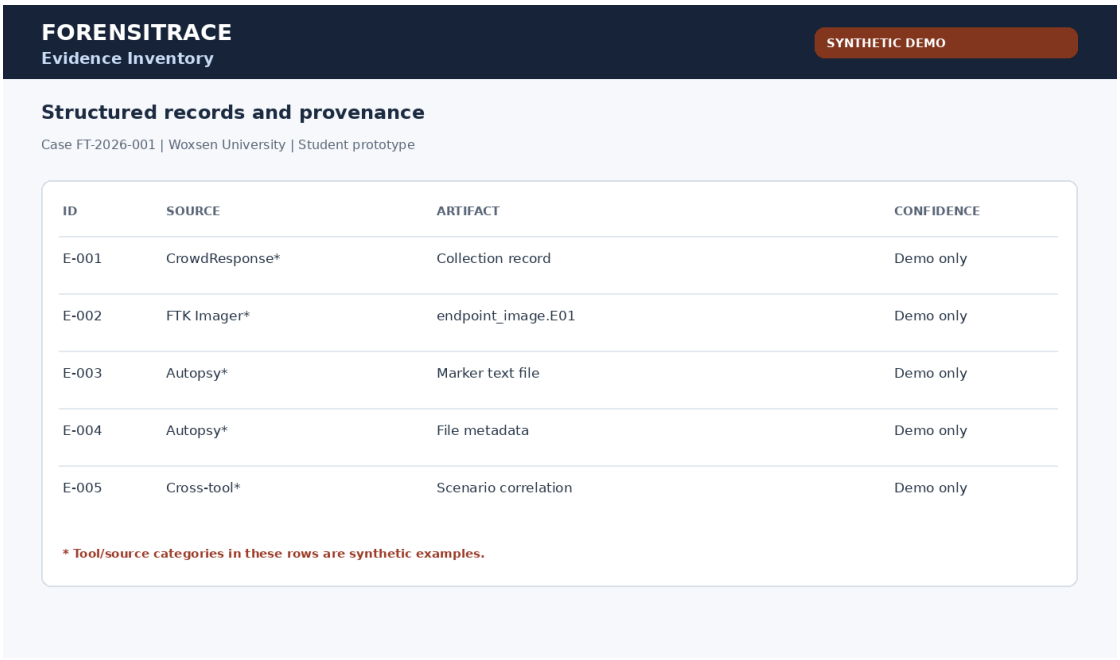


Figure 3.1: Evidence inventory illustration showing synthetic records and source categories.

3.3 Validation Logic

The Python utility performs these checks:

1. Loads the case and finding records from JSON files.
2. Checks whether required fields are present.
3. Parses timestamps using Python's ISO timestamp parser.
4. Detects duplicate evidence identifiers.
5. Produces a validation status and a list of any issues.
6. Exports a JSON correlation report and a CSV evidence table.

The validation status indicates structural consistency only. It does not prove that the source evidence is genuine, that a forensic tool operated correctly, or that an interpretation is causally correct.

3.4 Execution

From the project directory, execute:

```
python src/correlate_findings.py
```

```
python -m unittest discover -s tests -v
```

The utility writes `reports/correlation_report.json` and `reports/correlation_report.csv`. The included tests validate the demonstration dataset and check that a duplicate evidence ID is rejected.

CHAPTER 4

Results and Testing

4.1 Dataset Summary

The supplied synthetic dataset contains five findings with five unique evidence identifiers. The entries represent the types of records expected from live triage, disk acquisition, disk examination, and cross-source correlation. They are examples for testing the software and are not actual tool outputs.

4.2 Test Results

Test	Expected behaviour	Result
Synthetic dataset validation	Five records are loaded; required fields and timestamps are valid; evidence IDs are unique.	PASS
Duplicate-ID negative test	A duplicated evidence ID causes validation to fail.	PASS
Timestamp parsing	ISO-formatted timestamps are parsed successfully.	PASS
JSON report generation	The utility writes a structured correlation report.	PASS
CSV report generation	The utility exports findings in tabular form.	PASS
External forensic-tool execution	Actual output must be collected using the relevant tool in an authorized lab.	Not performed

Table 4.1: Results of the included software tests and the status of external-tool execution.

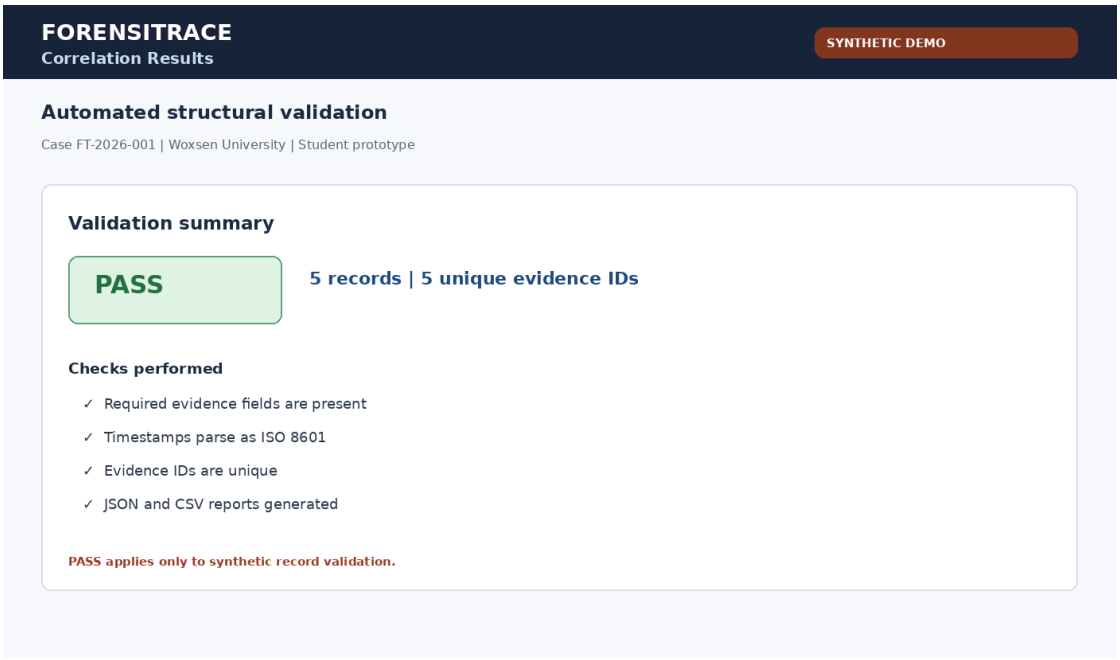


Figure 4.1: Correlation utility result illustration. PASS applies only to synthetic record validation.

4.3 Evidence Correlation Example

The synthetic example set contains a triage collection record, an acquisition record, an examination record, a metadata record, and a cross-source correlation record. The correlation utility stores them under unique IDs and checks their structure. Because no real tool outputs are included, the records cannot establish that an endpoint was compromised or that a real user performed a particular action.

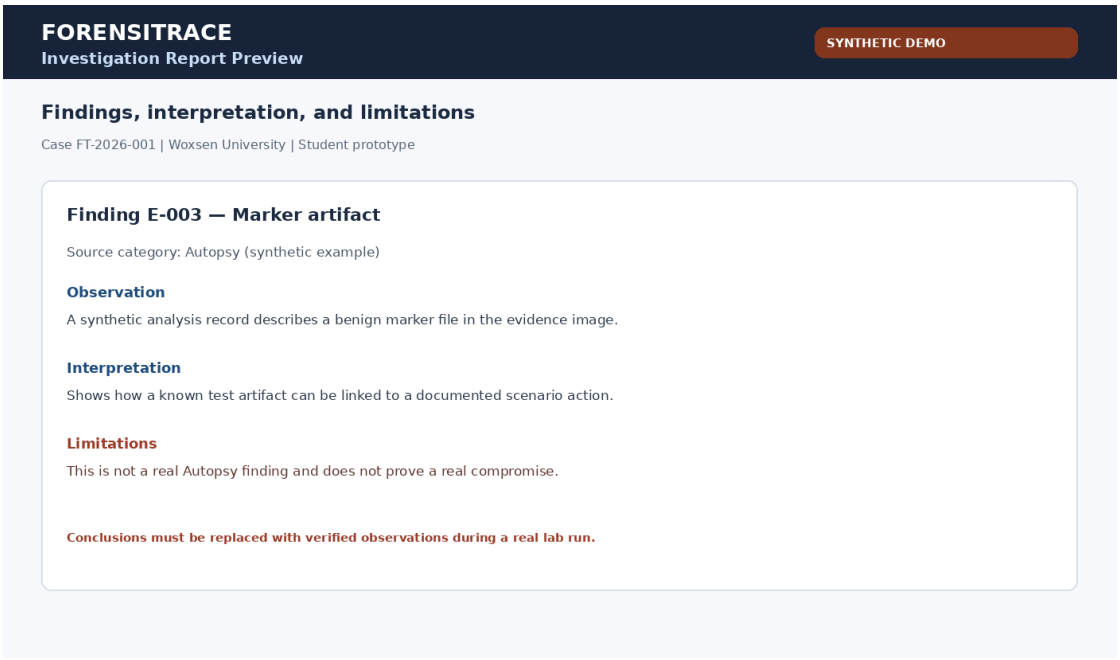


Figure 4.2: Report-preview illustration demonstrating the separation of observation, interpretation, and limitation.

4.4 Interpretation

The implementation demonstrates that findings from separate workflow stages can be organized into a consistent record format and exported for review. It also makes a useful distinction between what a record states and what an investigator infers from it. The results are limited to the behavior of the record-validation prototype.

CHAPTER 5

Evidence Handling and Limitations

5.1 Evidence Integrity

For a practical run, the investigator should preserve native tool outputs and record their origin. Forensic-image hashes should be copied from the actual acquisition and verification logs. A hash calculated over a project report or a synthetic JSON file is not a substitute for verifying a forensic image.

5.2 Chain of Custody

A chain-of-custody entry should include an item identifier, date and time, action, responsible person, source or destination, and integrity notes. The CSV included in the project records only the creation and validation of synthetic project data. It is not a chain-of-custody record for a seized device or actual forensic image.

5.3 Limitations

- The supplied findings are synthetic demonstration records.
- The screenshots are generated illustrations of the ForensiTrace interface, not screenshots captured from the three external forensic tools.
- The prototype checks record structure; it does not parse native output formats or independently validate tool results.
- No actual disk image or live-response collection is included.
- Timestamp interpretation depends on the source, time zone, clock accuracy, and filesystem behavior.
- Evidence consistency alone does not prove causation or establish a real security incident.

CHAPTER 6

Conclusion and Future Work

ForensiTrace demonstrates a structured way to organize evidence records across live triage, disk acquisition, and disk-image examination. The Python utility validates required fields, checks timestamp formatting, detects duplicate identifiers, and exports JSON and CSV reports. The included automated tests passed for the supplied synthetic dataset and rejected a duplicate-ID case.

The project establishes the documentation and correlation layer of the proposed workflow. A complete practical demonstration with CrowdResponse, FTK Imager, and Autopsy remains to be performed in an authorized Windows test environment. At that stage, synthetic examples should be replaced with actual outputs and genuine screenshots from the applications, with all conclusions tied to the verified evidence.

Future work could include parsers for selected native output formats, report generation from verified records, automated hashing of collected files, stronger schema validation, and a local interface for browsing cases and findings. Any such additions should preserve the original evidence and maintain a clear distinction between automated checks and examiner interpretation.

CHAPTER A

Project Execution and File Structure

A.1 Project Structure

```
ForensiTrace_Project/
  src/
    correlate_findings.py
  data/
    case.json
    findings.json
    evidence_inventory.json
  tests/
    test_correlation.py
  reports/
    correlation_report.json
    correlation_report.csv
    test_results.txt
  screenshots/
    01_case_overview.png
    02_evidence_inventory.png
    03_correlation_results.png
    04_report_preview.png
  docs/
    methodology.md
    chain_of_custody.csv
```

A.2 Reproduction Commands

```
python src/correlate_findings.py
python -m unittest discover -s tests -v
```

Python 3.10 or newer is recommended. The correlation prototype uses the Python standard library and requires no third-party Python packages.

A.3 Sample Finding

Evidence ID: E-003

Source category: Autopsy (synthetic example)

Artifact: `ForensicLab/DFIR_TEST_2026_01.txt`

Observation: A synthetic analysis record describes a benign marker file in an evidence image.

Interpretation: This illustrates how a known test artifact could be linked to a documented scenario action.

Limitation: This is not an actual Autopsy finding and does not prove a real incident.

References

1. CrowdResponse user documentation and configuration supplied with the laboratory materials. Consult the specific version used in the practical session.
2. FTK Imager documentation. Consult the version installed in the authorized lab environment for acquisition and image-verification procedures.
3. Autopsy and The Sleuth Kit documentation. Consult the version used for ingest configuration and disk-image examination.
4. Python Software Foundation, *Python 3 Documentation*, especially the `json`, `csv`, `datetime`, `argparse`, and `unittest` modules.